软件系统综合课程设计

练习 2 性能测试报告

稳定性测试

姓名： 程泽凯

项目名称： 西南交大校园论坛系统

测试主题： 稳定性测试

日期： 2026 年 6 月 17 日

软件工程实验室（X7507/7508）

1 实验目的与前提

本次报告选择稳定性测试作为个人负责内容。稳定性测试关注系统在持续请求下是否保持可用，是否出现错误率上升、响应时间突然变差或吞吐量明显下降。本轮实验受课堂时间限制，采用短时持续访问方式，不把 20 秒测试夸大成生产环境长稳压测。

目标部署服务器为 Ubuntu-mbpr，公网 IP 47.108.218.39，私有 IP 172.19.41.159，Ubuntu 22.04，2 核 CPU，2 GiB 内存，40 GiB 系统盘，公网带宽上限 200 Mbps。服务器监控快照显示 CPU 2.33%，内存 1.16 GiB/2 GiB，系统盘 8.57 GiB/40 GiB。本轮实测在本地同版本后端 demo profile 下完成，服务器信息用于说明部署约束。

2 测试场景总表

负责人	测试类型	测试内容	测试目标
金井直	负载测试	公共浏览混合访问	验证正常负载下是否稳定
蒋焰	压力测试	搜索与列表接口并发阶梯加压	观察高并发下响应时间增长
赵祺	容量测试	调整分页大小为 10、20、50、100	观察单页数据量扩大后的性能变化
程泽凯	稳定性测试	10 并发持续访问公共浏览接口 20 秒	检查短时持续运行中是否出现错误和抖动

3 测试环境

类别	配置
目标服务器	Ubuntu 22.04，2 核 CPU，2 GiB 内存，40 GiB 系统盘，200 Mbps 公网带宽
服务器快照	CPU 2.33%，内存 1.16 GiB/2 GiB，系统盘 8.57 GiB/40 GiB
本轮实测机	Windows 10.0.26200，16 逻辑 CPU，31.31 GiB 内存
被测服务	Spring Boot 3.2.0，JDK 17，demo profile，H2 内存数据库
测试地址	http://127.0.0.1:18081/api
测试工具	Node.js 并发请求脚本，固定 10 并发，持续混合请求 20 秒

4 性能指标要求

指标	要求	说明
错误率	0%	稳定性测试首先关注持续运行中是否出错
P95 响应时间	小于 200 ms	短时持续请求下不应出现大量慢请求
吞吐量	记录稳定区间实测值	观察持续访问中是否明显下降
资源利用率	记录服务器背景信息	本轮未采集长时间 CPU 曲线，不能伪造资源峰值

5 测试对象与统计口径

本轮稳定性测试选取公共浏览混合接口作为测试对象，关注重点并非瞬时最大吞吐，而是持续请求期间接口能否保持稳定返回。测试对象包含帖子列表、热门帖子、搜索和版块列表等读接口，这些接口可连续访问且不会改变业务状态，适合作为短时稳定性实验的基础场景。

统计口径包括成功率、错误率、吞吐量和响应时间分位数。成功率用于判断接口是否持续可用，错误率用于发现 5xx、连接失败或超时等异常。响应时间分位数用于观察请求是否出现长尾，尤其是 P95 和 P99。稳定性测试中，平均值不能单独说明问题，因为少量慢请求可能被平均值掩盖。

由于本轮测试持续时间较短，且仅包含一次完整样本，因此不使用置信区间。图表采用分位数柱状图和成功率/错误率百分比图。成功请求数与错误请求数差异极大，若直接使用原始请求数绘图，错误柱几乎不可见，因此采用百分比尺度更能准确表达结论。

6 实验执行过程

实验执行前，先确认后端服务及公共接口均处于可访问状态。随后启动稳定性测试脚本，以固定并发持续循环访问接口集合。测试期间不重启后端服务，也不手工清理缓存，以便观察系统在连续请求条件下的自然表现。

测试结束后，脚本统计总请求数、成功请求数、错误请求数、吞吐量和响应时间分布。本轮实验属于课程阶段的短时稳定性验证，能够发现接口立即报错、连接不稳定或响应时间明显恶化等问题，但不能替代长时间稳定性测试。因此，结论限定为短时持续访问稳定，而不扩展为线上长期稳定。

7 稳定性测试方案设计

测试采用 10 个并发 worker，持续 20 秒循环访问公共接口。接口组合包括帖子列表、热门帖子、关键词搜索和版块列表。选择这些接口的原因是它们都属于论坛首页或公共浏览链路，访问频率高，而且不需要登录态，适合做稳定性基础验证。

序号	接口	说明
1	/posts?page=0&size=20	首页帖子列表
2	/posts/hot?page=0&size=20	热门帖子列表
3	/posts/search?keyword= 测试 &page=0&size=20	关键词搜索
4	/sections?page=0&size=20	版块列表

8 测试结果展示

场景	并发	持续时间	总请求	吞吐量	错误率
STAB-01 短时持续混合访问	10	20 秒	60825	3041.10 req/s	0.00%

平均响应	P50	P90	P95	P99
3.29 ms	3.05 ms	4.44 ms	5.34 ms	7.18 ms

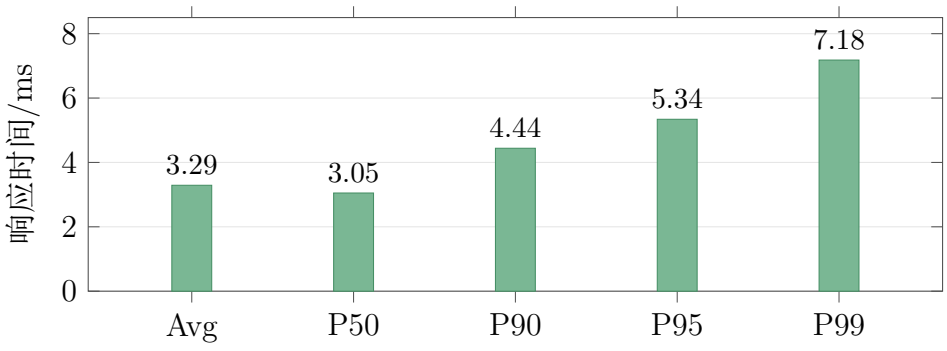


图 1: 稳定性测试响应时间分布

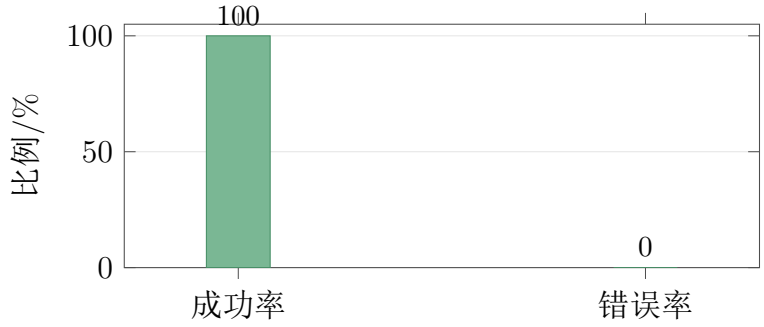


图 2: 稳定性测试成功率与错误率

从图表表达与测试目标看，稳定性测试关注请求耗时是否集中以及是否存在长尾，因此采用 Avg、P50、P90、P95、P99 展示响应时间分布。各分位数均处于毫秒级同一数量级，线性坐标即可表达差异，无需使用对数坐标或断裂轴。

成功与错误结果未继续使用原始请求数绘图。原始成功请求为 60825，错误请求为 0，二者差异过大；若使用请求数柱状图，错误柱几乎不可见，图表信息量较低。因此报告改用成功率与错误率百分比图，直接表达本轮持续访问未出现错误的结论。该处理比断裂坐标更简洁，也更符合稳定性测试的结论重点。

从响应时间看，平均响应时间为 3.29 ms，P95 为 5.34 ms，P99 为 7.18 ms，响应时间分布较集中。最大响应时间为 17.88 ms，没有出现明显长尾。吞吐量为 3041.10 req/s，说明本地 demo 环境下短时间持续请求比较平稳。

从资源角度看，目标云服务器只有 2 GiB 内存，而监控快照已使用 1.16 GiB。正式稳定性测试应至少持续 30 分钟到 2 小时，并同步记录 CPU、内存、GC、数据库连接池和网络带宽。本轮结果只能证明同版本后端在本地短时持续访问下稳定，不能替代线上长稳压测。

9 测试有效性与局限

本次稳定性测试的有效性在于请求数量较多，并且使用混合公共接口持续访问，能够基本验证系统在短时间内不会因为重复请求而立即出错。测试结果同时记录了成功请求数、错误请求数、吞吐量和响应时间分位数，便于判断是否存在明显异常。

局限性是持续时间不足，且没有按时间窗口拆分结果。如果只给出 20 秒整体平均值，可能掩盖前 5 秒和后 5 秒的差异。例如系统可能在开始阶段较慢，后续因缓存变快；也可能前期稳定，后期逐渐变慢。当前报告无法回答这种趋势问题。另外，本轮没有监控 JVM 堆、GC、线程数和数据库连接池，也没有检查测试结束后服务日志，因此稳定性结论仍属于短时验证。

后续改进应把稳定性测试拆成多个时间窗口，例如每 30 秒统计一次 P95、错误率和吞吐量，并绘制趋势图。如果连续多个窗口指标保持接近，稳定性结论会更有说服力；如

果某个窗口开始升高，就可以结合日志和资源监控定位原因。

10 缺陷与风险记录

编号	现象或风险	处理建议
STAB-R1	20 秒持续时间较短，不能证明长期稳定	增加 30 分钟以上持续测试，并分窗口统计结果
STAB-R2	未采集 JVM 和系统资源，无法判断稳定性余量	同步记录 CPU、内存、堆使用、GC 和线程数
STAB-R3	只覆盖公共读接口，未触发写入和登录相关状态变化	增加登录、发帖、评论等混合业务流

11 优化方案

1. 上云部署后增加长期监控，重点看 JVM 堆、GC 次数、线程数、数据库连接池和系统内存。
2. 对公共 GET 接口增加缓存，降低持续访问时数据库压力。
3. 日志级别不要长期保持 DEBUG，避免稳定性测试时日志写入成为额外瓶颈。
4. 稳定性测试正式执行时应增加 30 分钟、1 小时、2 小时三个时长档位，观察是否存在内存增长趋势。

12 同项目性能观测点对比

测试类型	代表场景	并发/容量	吞吐量	P95	结论
负载测试	混合浏览	8 并发	1147.24	15.78 ms	正常负载可用
压力测试	搜索 + 列表	48 并发	2590.05	27.59 ms	高并发仍无错误
容量测试	page size=100	12 并发	2337.80	7.95 ms	大分页吞吐下降
稳定性测试	混合持续访问	10 并发	3041.10	5.34 ms	短时持续访问稳定

13 实验收获

本次稳定性测试表明，短时间内响应较快并不等同于系统具备长期稳定性。本轮实验持续时间为 20 秒，更适合作为课程阶段的短时稳定性验证，用于检查公共接口在连续访问下是否出现即时错误或明显抖动。

若要评估真实部署环境中的长期稳定性，应增加持续测试时间，并同步观察内存增长、线程数量、GC 情况、数据库连接池和日志体积变化。本次报告对测试时长和环境边界作出明确说明，能够避免将短时实验结果扩大解释为线上长期稳定结论。

A 性能测试脚本完整代码

本附录给出本次报告使用的完整 Node.js 性能测试脚本。脚本负责启动后端 demo 环境、等待接口可用，并依次执行四类性能测试。测试结束后会输出 JSON 和 CSV 两种结果文件。代码块中的注释用于说明关键步骤，便于复现实验过程。

```
// Import Node built-in modules for file I/O, OS metadata, paths, and process
↪  spawning.
import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import { spawn } from "node:child_process";

// Resolve the backend artifact path and the local API base URL.
const root = path.resolve("../../");
const backend = path.join(root, "jiaodaluntan", "backend");
const jar = path.join(backend, "target", "forum-1.0.0.jar");
const port = Number(process.env.PERF_PORT || 18081);
const baseUrl = `http://127.0.0.1:${port}/api`;

// Start Spring Boot with demo profile and an in-memory H2 database for
↪  repeatable tests.
const javaArgs = [
  "-jar",
  jar,
  "--spring.profiles.active=demo",
  `--server.port=${port}`,
  "--spring.datasource.url=jdbc:h2:mem:swjtu_perf;MODE=MySQL;DATABASE_TO_LOW」
  ↪  ER=TRUE;CASE_INSENSITIVE_IDENTIFIERS=TRUE;DB_CLOSE_DELAY=-1;DB_CLOSE_O」
  ↪  N_EXIT=FALSE",
  "--demo.seed.enabled=true",
  "--demo.seed.run-on-demo-profile=true",
];

// Spawn the backend process in the background; the script stops it after
↪  testing.
const app = spawn("java", javaArgs, {
  cwd: backend,
  stdio: ["ignore", "ignore", "ignore"],
  windowsHide: true,
```

```
});

const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// Probe the backend until it is ready to accept requests.
async function probe() {
  try {
    const response = await fetch(`${baseUrl}/posts?page=0&size=1`, {
      signal: AbortSignal.timeout(2000),
    });
    return response.status < 500;
  } catch {
    return false;
  }
}

// Calculate percentile values from sorted successful response times.
function percentile(sorted, p) {
  if (!sorted.length) return 0;
  const index = Math.ceil((p / 100) * sorted.length) - 1;
  return sorted[Math.max(0, Math.min(sorted.length - 1, index))];
}

// Convert raw request records into report metrics such as P95, throughput,
↪  and error rate.
function summarize(name, type, endpointLabel, records, durationMs,
↪  concurrency, extra = {}) {
  const ok = records.filter((record) => record.ok);
  const errors = records.length - ok.length;
  const times = ok.map((record) => record.ms).sort((a, b) => a - b);
  const avg = times.reduce((sum, value) => sum + value, 0) / (times.length ||
  ↪  1);
  return {
    name,
    type,
    endpointLabel,
    concurrency,
    durationMs,
    total: records.length,
    success: ok.length,
```

```
    errors,
    errorRate: records.length ? errors / records.length : 0,
    throughput: records.length / (durationMs / 1000),
    avg,
    p50: percentile(times, 50),
    p90: percentile(times, 90),
    p95: percentile(times, 95),
    p99: percentile(times, 99),
    max: times[times.length - 1] || 0,
    ...extra,
  };
}

// Execute one HTTP request and record status plus elapsed time.
async function oneRequest(url) {
  const start = performance.now();
  try {
    const response = await fetch(baseUrl + url, {
      signal: AbortSignal.timeout(5000),
    });
    await response.arrayBuffer();
    return { ok: response.ok, status: response.status, ms: performance.now()
    ↪  - start };
  } catch (error) {
    return { ok: false, status: 0, ms: performance.now() - start, error:
    ↪  String(error.message || error) };
  }
}

// Run a fixed-concurrency scenario until the configured duration expires.
async function runFixed(name, type, urls, concurrency, seconds, extra = {}) {
  const endAt = Date.now() + seconds * 1000;
  const records = [];
  let index = 0;
  async function worker() {
    while (Date.now() < endAt) {
      records.push(await oneRequest(urls[index++ % urls.length]));
    }
  }
  const start = Date.now();
```

```
  await Promise.all(Array.from({ length: concurrency }, worker));
  return summarize(name, type, urls.join(" | "), records, Date.now() - start,
  ↪ concurrency, extra);
}

// Main workflow: wait for backend, run four performance scenarios, and write
↪ result files.
async function main() {
  let started = false;
  for (let i = 0; i < 90; i += 1) {
    if (await probe()) {
      started = true;
      break;
    }
    await sleep(1000);
  }
  if (!started) {
    app.kill();
    throw new Error("Backend did not start in 90 seconds");
  }

  // Mixed public browsing workload used by load and stability tests.
  const mix = [
    "/posts?page=0&size=20",
    "/posts/hot?page=0&size=20",
    "/posts/search?keyword= 测试 &page=0&size=20",
    "/sections?page=0&size=20",
  ];

  const results = [];
  // Load test: normal mixed browsing at moderate concurrency.
  results.push(await runFixed("LOAD-01 常规混合浏览负载", "负载测试", mix, 8,
  ↪ 12, { phase: "baseline" }));
  // Stress test: increase concurrency step by step.
  for (const concurrency of [8, 16, 32, 48]) {
    results.push(await runFixed(
      `STRESS-${concurrency} 并发阶梯`,
      " 压力测试",
      ["/posts/search?keyword=测试&page=0&size=20", "/posts?page=0&size=20"],
      concurrency,
```

```
    6,
    { level: concurrency },
  ));
}
// Capacity test: increase page size while keeping concurrency fixed.
for (const pageSize of [10, 20, 50, 100]) {
  results.push(await runFixed(
    `CAP-${pageSize} 分页容量`,
    " 容量测试",
    [`/posts?page=0&size=${pageSize}`,
    ↪  `/posts/search?keyword=测试&page=0&size=${pageSize}`],
    12,
    5,
    { pageSize },
  ));
}
// Stability test: short continuous mixed workload.
results.push(await runFixed("STAB-01 短时持续混合访问", "稳定性测试", mix,
↪  10, 20, { phase: "short-soak" }));

app.kill();

// Persist machine metadata and measured results for report tables and
↪  charts.
const payload = {
  meta: {
    generatedAt: new Date().toISOString(),
    baseUrl,
    host: {
      platform: os.platform(),
      release: os.release(),
      cpus: os.cpus().length,
      totalMemGiB: Number((os.totalmem() / 1024 / 1024 / 1024).toFixed(2)),
    },
    serverInfoFromUser: {
      publicIp: "47.108.218.39",
      privateIp: "172.19.41.159",
      os: "Ubuntu 22.04",
      cpu: "2 核",
      memory: "2 GiB",
```

```
        disk: "40 GiB",
        bandwidth: "200 Mbps",
        region: " 西南 1（成都）",
      },
    },
    results,
  };

  // Write JSON and CSV outputs so the report can cite reproducible raw data.
  fs.writeFileSync("performance_results.json", JSON.stringify(payload, null,
  ↪ 2), "utf8");
  const csv = [
    "name,type,concurrency,total,success,errors,errorRate,throughput,avg,p50」
    ↪ ,p90,p95,p99,max,endpointLabel",
    ...results.map((r) => [
      r.name,
      r.type,
      r.concurrency,
      r.total,
      r.success,
      r.errors,
      r.errorRate.toFixed(4),
      r.throughput.toFixed(2),
      r.avg.toFixed(2),
      r.p50.toFixed(2),
      r.p90.toFixed(2),
      r.p95.toFixed(2),
      r.p99.toFixed(2),
      r.max.toFixed(2),
      `"${r.endpointLabel.replaceAll('"', '""')}"`,
    ].join(",")),
  ];
  fs.writeFileSync("performance_results.csv", csv.join("\n"), "utf8");
  console.log(`Wrote ${results.length} performance result rows.`);
}

// Ensure backend process is stopped even when a test step fails.
main().catch((error) => {
  app.kill();
  console.error(error);
```

```
  process.exitCode = 1;
});
```